

GESTIONNAIRE DE MOTS DE PASSE CHIFFRÉ

TP ÉLECTRONIQUE — MONTAGE ET CÂBLAGE SUR TINKERCAD

Classe : _____

Nom / Prénom : _____

Date : _____

PLANNING DU TP (3 HEURES)

- | | |
|--|---------|
| ■ Partie 1 — Découverte des composants | ~30 min |
| ■ Partie 2 — Calcul et protection des LEDs (Loi d'Ohm) | ~30 min |
| ■ Partie 3 — Câblage progressif du circuit sur Tinkercad | ~50 min |
| ■ Partie 4 — Vérification, tests et dépannage | ~25 min |
| ■ Partie 5 — Questions d'analyse et de synthèse | ~35 min |

Objectif final : Avoir un circuit complet et fonctionnel sur Tinkercad !

OBJECTIFS DU TP

À la fin de ce TP, vous serez capables de :

- Identifier et décrire le rôle de chaque composant électronique du projet
- Appliquer la loi d'Ohm pour calculer la valeur des résistances
- Réaliser le câblage complet du circuit sur Tinkercad
- Tester et valider le fonctionnement de chaque composant
- Expliquer le principe de fonctionnement d'un clavier matriciel et du protocole I2C

PARTIE 1 — DÉCOUVERTE DES COMPOSANTS (~30 min)

DURÉE ESTIMÉE : 30 MINUTES

Avant de câbler quoi que ce soit, il est essentiel de bien connaître les composants que vous allez utiliser. Dans cette partie, vous allez les identifier, les observer et remplir des tableaux de caractéristiques.

1.1 Liste du matériel — Vérification

Ouvrez Tinkercad et créez un nouveau circuit. Vérifiez que vous pouvez trouver chacun des composants ci-dessous dans la bibliothèque. Cochez la case correspondante :

✓	Composant	Quantité	Trouvé dans Tinkercad ?	Prix indicatif	Prix (sur le marché) ?
☐	Arduino Uno R3	1	☐ Oui ☐ Non	~25€	
☐	Clavier matriciel 4×4	1	☐ Oui ☐ Non	~5€	
☐	Écran LCD 16×2 avec module I2C	1	☐ Oui ☐ Non	~8€	
☐	LED verte	1	☐ Oui ☐ Non	~0.20€	
☐	LED bleue	1	☐ Oui ☐ Non	~0.20€	
☐	LED jaune	1	☐ Oui ☐ Non	~0.20€	
☐	Résistance 220 Ω	3	☐ Oui ☐ Non	~0.10€	
☐	Buzzer actif 5V	1	☐ Oui ☐ Non	~2€	

1.2 Identification des composants — Tableau à compléter

Complétez le tableau suivant en observant chaque composant dans Tinkercad (passez la souris dessus pour voir les informations, ou consultez vos ressources) :

Composant	Rôle dans le projet	Nb de broches / connexions	Type : Entrée ou Sortie ?
Arduino Uno R3			
Clavier matriciel 4×4			
Écran LCD 16×2 I2C			
LED (Verte / Bleue / Jaune)			
Résistance 220 Ω			
Buzzer actif 5V			

1.3 La LED — Anatomie et polarité

RAPPEL : LA DIODE ÉLECTROLUMINESCENTE (LED)

Une LED laisse passer le courant dans un seul sens et émet de la lumière quand elle est traversée. Elle possède 2 broches :

- L'Anode (+) : la patte LONGUE — côté positif
 - La Cathode (-) : la patte COURTE — côté négatif (souvent côté méplat)
- ⚠ SENS IMPORTANT : le courant entre par l'anode et sort par la cathode.
Si vous la branchez à l'envers, elle ne s'allume pas (et ne casse pas).

? QUESTION A

Sur le schéma d'une LED ci-dessous (à dessiner dans l'espace prévu), indiquez par des flèches : l'anode (+), la cathode (-), et le sens du courant. Précisez aussi quelle patte est la plus longue.

[Espace pour le schéma — 8 cm de hauteur]

PARTIE 2 — CALCUL ET PROTECTION DES LEDs (~30 min)**DURÉE ESTIMÉE : 30 MINUTES**

Une LED ne peut pas être connectée directement à l'Arduino sans résistance ! Dans cette partie, vous allez comprendre pourquoi, et apprendre à calculer la valeur correcte d'une résistance grâce à la Loi d'Ohm.

2.1 La Loi d'Ohm — Rappel**LOI D'OHM**

La loi d'Ohm relie la tension (U), le courant (I) et la résistance (R) :

$$U = R \times I$$

Où :

- U est la tension en Volts (V)
- I est le courant en Ampères (A)
- R est la résistance en Ohms (Ω)

On peut aussi l'écrire : $R = U / I$ ou $I = U / R$

2.2 Pourquoi une résistance est-elle obligatoire ?

Une LED a une tension de fonctionnement typique de 2V et un courant maximum de 20 mA (0.020 A). L'Arduino fournit 5V sur ses pins numériques.

⚠ SANS RÉSISTANCE

Sans résistance, la LED serait soumise à 5V alors qu'elle ne supporte que 2V.

- Le courant serait beaucoup trop élevé.
- La LED grille immédiatement (en quelques millisecondes).
- L'Arduino peut également être endommagé (courant max par pin : 40 mA).

2.3 Calcul de la résistance — Application numérique**MÉTHODE DE CALCUL**

La résistance doit absorber la différence de tension entre l'Arduino (5V) et la LED (2V).

1. Calculer la tension aux bornes de la résistance :

$$U_R = U_{\text{Arduino}} - U_{\text{LED}} = 5V - 2V = 3V$$

2. Choisir le courant de travail (on prend 15 mA = 0.015 A pour une bonne luminosité) :

$$I = 15 \text{ mA} = 0.015 \text{ A}$$

3. Appliquer la Loi d'Ohm :

$$R = U_R / I = 3V / 0.015A = 200 \Omega$$

4. Choisir la résistance commerciale supérieure la plus proche : 220 Ω ✓

(Une résistance plus grande protège mieux, mais la LED sera un peu moins lumineuse.)

? QUESTION B

Exercice de calcul : Si on voulait alimenter la LED avec un courant de 10 mA au lieu de 15 mA, quelle serait la valeur idéale de la résistance ? Montrez votre calcul complet.

Données : $U_{\text{Arduino}} = 5\text{V}$, $U_{\text{LED}} = 2\text{V}$, $I = 10\text{ mA} = 0.010\text{ A}$

? QUESTION C

Regardez le code couleur d'une résistance de 220 Ω . Quelles sont les 4 bandes de couleur ? (Indice : Rouge - Rouge - Marron - Or). Expliquez comment lire ce code couleur.

Bande 1 (centaines)	Bande 2 (dizaines)	Bande 3 (multiplicateur)	Bande 4 (tolérance)

2.4 Schéma du circuit LED à compléter

Complétez le schéma de câblage d'une seule LED (LED verte, Pin 13) en ajoutant dans les cases les valeurs manquantes :

Arduino Pin 13 $\rightarrow$ [Ω] $\rightarrow$ **Anode (+)** ☐ **Cathode (-)** $\rightarrow$ **GND**
(5V) résistance LED verte (0V)

Complétez : La résistance vaut _____ Ω . La tension aux bornes de la LED est _____ V. La tension aux bornes de la résistance est _____ V.

PARTIE 3 — CÂBLAGE PROGRESSIF SUR TINKERCAD (~50 min)**DURÉE ESTIMÉE : 50 MINUTES**

Vous allez câbler le circuit en plusieurs étapes, du plus simple au plus complexe. Faites valider chaque étape par votre professeur avant de continuer.

🔗 CONSEILS AVANT DE COMMENCER

- Lisez entièrement chaque étape AVANT de commencer le câblage
- Utilisez des couleurs différentes pour chaque fil (rouge = 5V, noir = GND, autres couleurs = signaux)
- Vérifiez vos connexions une par une avec le tableau de câblage
- En cas de doute, demandez à votre professeur

ÉTAPE 1 — Câblage des LEDs et du Buzzer (environ 15 min)**Tableau de câblage — LEDs et Buzzer**

Composant	Broche composant	Connexion intermédiaire	Pin Arduino	✓ Fait ?
LED Verte	Anode (+) patte longue	→ Résistance 220 Ω →	Pin 13	☐
LED Verte	Cathode (-) patte courte	—	GND	☐
LED Bleue	Anode (+) patte longue	→ Résistance 220 Ω →	Pin 12	☐
LED Bleue	Cathode (-) patte courte	—	GND	☐
LED Jaune	Anode (+) patte longue	→ Résistance 220 Ω →	Pin 11	☐
LED Jaune	Cathode (-) patte courte	—	GND	☐
Buzzer actif	Broche + (anode)	—	Pin 10	☐
Buzzer actif	Broche - (cathode)	—	GND	☐

☑ POINT DE VALIDATION 1 — LEDs et Buzzer

Avant de continuer, vérifiez les points suivants :

- ☐ Les 3 LEDs sont câblées avec leur résistance 220 Ω en série
- ☐ Toutes les cathodes (-) des LEDs sont reliées au GND
- ☐ Le buzzer est câblé correctement (+ → Pin 10, - → GND)
- ☐ Aucun fil ne se croise ou ne touche un mauvais composant

→ Faites valider par votre professeur avant de passer à l'étape 2 ✓

Signature du professeur : _____

ÉTAPE 2 — Câblage du clavier matriciel 4×4 (environ 15 min)**🔑 PRINCIPE DU CLAVIER MATRICIEL**

Un clavier 4×4 possède 16 touches mais seulement 8 fils de connexion !

Comment est-ce possible ? Grâce à une organisation en lignes et colonnes :

- 4 fils pour les LIGNES (R1 à R4) — connectés en sorties Arduino
- 4 fils pour les COLONNES (C1 à C4) — connectés en entrées Arduino

L'Arduino allume une ligne à la fois (à l'état BAS), puis lit les colonnes.

Si une touche est pressée à l'intersection de la ligne active et d'une colonne, la colonne correspondante passe à l'état BAS → la touche est identifiée !

Ce mécanisme s'appelle le balayage matriciel (scanning).

Tableau de câblage — Clavier 4×4

Broche clavier	Description	Pin Arduino	✓ Fait ?
R1	Ligne 1 (rangée du haut : 1, 2, 3, A)	Pin 9	☐
R2	Ligne 2 (rangée : 4, 5, 6, B)	Pin 8	☐
R3	Ligne 3 (rangée : 7, 8, 9, C)	Pin 7	☐
R4	Ligne 4 (rangée du bas : *, 0, #, D)	Pin 6	☐
C1	Colonne 1 (touches : 1, 4, 7, *)	Pin 5	☐
C2	Colonne 2 (touches : 2, 5, 8, 0)	Pin 4	☐
C3	Colonne 3 (touches : 3, 6, 9, #)	Pin 3	☐
C4	Colonne 4 (touches : A, B, C, D)	Pin 2	☐

? QUESTION D

Combien de fils faudrait-il si le clavier n'utilisait PAS le système matriciel (une connexion directe par touche + un fil commun) ? Faites le calcul : ____ touches × ____ fils = ____ fils au total. Comparez avec les ____ fils utilisés grâce à la matrice.

✓ POINT DE VALIDATION 2 — Clavier matriciel

Vérifiez les points suivants avant de continuer :

- ☐ Les 4 fils de lignes (R1-R4) sont connectés aux Pins 9-8-7-6
- ☐ Les 4 fils de colonnes (C1-C4) sont connectés aux Pins 5-4-3-2
- ☐ L'ordre des fils est correct (R1→Pin9, R2→Pin8, R3→Pin7, R4→Pin6)
- ☐ Le clavier est orienté correctement dans Tinkercad

→ Faites valider par votre professeur avant de passer à l'étape 3 ✓

Signature du professeur : _____

ÉTAPE 3 — Câblage de l'écran LCD 16×2 avec module I2C (environ 15 min)

LE PROTOCOLE I2C — COMPRENDRE L'ESSENTIEL

I2C (Inter-Integrated Circuit) est un protocole de communication série inventé par Philips.

Il permet de connecter plusieurs périphériques avec seulement 2 fils de données :

- SDA (Serial Data) : le fil qui transporte les données
- SCL (Serial Clock) : le fil d'horloge qui synchronise la communication

Avantage majeur : un écran LCD classique (sans I2C) nécessite 6 à 10 fils !

Avec le module I2C, il n'en faut que 4 (SDA, SCL, 5V, GND).

Comment ça marche ?

1. Chaque composant I2C possède une adresse unique (ex: 0x27 ou 0x3F)
2. L'Arduino envoie d'abord l'adresse du composant visé
3. Puis envoie les données bit par bit sur SDA, synchronisé par SCL
4. Plusieurs composants I2C peuvent partager les mêmes 2 fils (bus I2C) !

Tableau de câblage — LCD I2C

Broche LCD I2C	Fonction	Connexion Arduino	✓ Fait ?
VCC	Alimentation positive	Broche 5V de l'Arduino	☐
GND	Masse (0V)	Broche GND de l'Arduino	☐
SDA	Données I2C	Pin A4 (SDA) de l'Arduino	☐
SCL	Horloge I2C	Pin A5 (SCL) de l'Arduino	☐

ADRESSE I2C DE L'ÉCRAN

L'adresse I2C par défaut du module est 0x27 (en hexadécimal).

Certains modules utilisent l'adresse 0x3F — à vérifier si l'écran ne fonctionne pas.

Dans Tinkercad, l'adresse 0x27 fonctionne dans la plupart des cas.

Dans le code Arduino : `LiquidCrystal_I2C lcd(0x27, 16, 2);`

↑ ↑ ↑
 adresse colonnes lignes

POINT DE VALIDATION 3 — Écran LCD I2C

Vérifiez les points suivants :

- ☐ VCC du LCD → 5V de l'Arduino (fil rouge)
- ☐ GND du LCD → GND de l'Arduino (fil noir)
- ☐ SDA du LCD → Pin A4 de l'Arduino
- ☐ SCL du LCD → Pin A5 de l'Arduino

→ Faites valider par votre professeur. Circuit complet ! ✓

Signature du professeur : _____

ÉTAPE 4 — Récapitulatif des connexions

Maintenant que tout est câblé, complétez ce tableau de récapitulatif :

Composant	Pins Arduino utilisés	Nombre de fils
Clavier matriciel 4×4	Pins 2, 3, 4, 5, 6, 7, 8, 9	8 fils
Écran LCD 16×2 I2C		
LED Verte + Résistance		
LED Bleue + Résistance		
LED Jaune + Résistance		
Buzzer actif		
TOTAL		

PARTIE 4 — VÉRIFICATION, TESTS ET DÉPANNAGE (~25 min)**DURÉE ESTIMÉE : 25 MINUTES****4.1 Test de chaque composant avec un mini-code**

Pour vérifier que vos connexions sont correctes, testez chaque composant séparément avec de petits programmes Arduino. Demandez à votre professeur le lien vers les codes de test.

Composant	Résultat attendu du test	Résultat obtenu
LED Verte (Pin 13)	La LED verte s'allume et clignote 1 fois/seconde	☐ OK ☐ Problème
LED Bleue (Pin 12)	La LED bleue s'allume et clignote 1 fois/seconde	☐ OK ☐ Problème
LED Jaune (Pin 11)	La LED jaune s'allume et clignote 1 fois/seconde	☐ OK ☐ Problème
Buzzer (Pin 10)	Le buzzer émet un bip régulier	☐ OK ☐ Problème
Écran LCD I2C	L'écran affiche "Bonjour !" sur la ligne 1	☐ OK ☐ Problème
Clavier matriciel	La touche pressée s'affiche sur le moniteur série	☐ OK ☐ Problème

4.2 Guide de dépannage — Si un composant ne fonctionne pas**🔧 MÉTHODE DE DÉPANNAGE SYSTÉMATIQUE**

Si un composant ne fonctionne pas, suivez ces étapes dans l'ordre :

1. CÂBLAGE : Revérifiez chaque connexion une par une avec le tableau de câblage
2. POLARITÉ : Pour les LEDs et le buzzer, vérifiez que l'anode est du bon côté
3. PIN NUMBER : Vérifiez que le numéro de pin dans le code correspond au câblage
4. ALIMENTATION : Vérifiez que 5V et GND sont bien connectés
5. COMPOSANT : Essayez un autre composant du même type (LED différente...)

Symptôme observé	Cause probable	Solution
La LED ne s'allume pas du tout	LED branchée à l'envers (polarité inversée)	Inverser l'anode et la cathode
La LED est très faible (peine à briller)	Résistance trop grande ou mauvais pin	Vérifier la valeur de R et le numéro de pin
L'écran LCD affiche des cases noires	Mauvaise adresse I2C (0x27 ou 0x3F)	Tester les deux adresses dans le code
L'écran LCD n'affiche rien	SDA/SCL inversés ou mauvaise alimentation	Vérifier A4=SDA et A5=SCL, et 5V/GND
Le clavier ne répond pas	Pins de lignes/colonnes mal branchés	Revérifier R1-R4 (Pins 9-6) et C1-C4 (Pins 5-2)
Mauvaise touche affichée	Ordre des fils R/C inversé	Vérifier l'ordre exact des connexions

4.3 Note de dépannage — Espace de compte-rendu

Si vous avez rencontré un problème, décrivez-le ici et notez comment vous l'avez résolu :

Composant défaillant	Problème observé	Solution trouvée

PARTIE 5 — QUESTIONS D'ANALYSE ET DE SYNTHÈSE (~35 min)**DURÉE ESTIMÉE : 35 MINUTES**

Ces questions vous permettent de vérifier et d'approfondir votre compréhension de l'électronique du projet. Répondez de manière précise et argumentée.

5.1 Questions sur les LEDs et la protection**? QUESTION 1**

Combien de pins Arduino sont utilisés uniquement pour les LEDs et le buzzer ? Listez-les tous. Quelle est la différence entre le câblage d'une LED et celui du buzzer (en ce qui concerne la résistance) ?

? QUESTION 2

Pourquoi est-il obligatoire de mettre une résistance de $220\ \Omega$ en série avec chaque LED ? Que se passerait-il si on n'en mettait pas ? Donnez une réponse complète avec des valeurs numériques.

? QUESTION 3

Dans ce projet, les LEDs sont connectées entre un pin Arduino et le GND. Expliquez ce qu'il se passe au niveau électronique quand l'Arduino met le pin à l'état HAUT (HIGH) : que vaut la tension sur le pin ? Comment le courant circule-t-il ? Quelle est la tension aux bornes de la LED ? Et aux bornes de la résistance ?

5.2 Questions sur le clavier matriciel

? QUESTION 4

Expliquez en détail le principe du clavier matriciel 4×4. Comment l'Arduino sait-il quelle touche a été pressée ? Décrivez le mécanisme de balayage des lignes et colonnes.

? QUESTION 5

Le clavier possède 16 touches mais seulement 8 fils. Calculez l'économie de fils par rapport à une connexion directe (1 fil par touche + 1 fil commun). Exprimez également ce gain en pourcentage : combien de fils évite-t-on ?

5.3 Questions sur le protocole I2C

? QUESTION 6

Quel est l'avantage principal du module I2C pour l'écran LCD par rapport à une connexion parallèle classique ? Combien de pins économise-t-on ? Combien de pins faudrait-il sans le module I2C (connexion directe LCD standard) ?

? QUESTION 7

L'adresse I2C du LCD est 0x27. À quoi sert cette adresse ? Que se passerait-il si deux composants I2C avaient la même adresse sur le même bus ? Dans quel autre domaine connaissez-vous un système d'adresses similaire ?

5.4 Question de synthèse**? QUESTION 8**

Regardez le récapitulatif des connexions du projet (20 pins utilisés sur 26 disponibles). Quels sont les 6 pins qui restent libres sur l'Arduino Uno ? Si on voulait ajouter une LED rouge supplémentaire pour signaler une erreur, quel pin pourriez-vous utiliser ? Justifiez votre réponse.

? QUESTION 9

Imaginez que vous devez expliquer ce circuit électronique à un camarade qui n'a jamais fait d'électronique. Rédigez un texte de 6 à 10 lignes décrivant ce que fait chaque composant et comment ils communiquent ensemble pour former le système complet.

PARTIE 6 — LECTURE DE DATASHEETS (Approfondissement)

C'EST QUOI UNE DATASHEET ?

Une datasheet (fiche technique) est le document officiel d'un composant électronique.

Rédigée par le fabricant, elle contient TOUTES les informations techniques : tensions limites, courants max, températures, délais, schémas internes, timing...

Un bon technicien en électronique DOIT savoir lire une datasheet.

C'est la source la plus fiable — bien plus qu'un tutoriel ou un forum.

Pour trouver une datasheet : tapez le nom du composant + 'datasheet' sur Google.

Exemple : '5mm red LED datasheet' / 'ATmega328P datasheet' / 'PCF8574 datasheet'

6.1 Datasheet de la LED 5mm — Valeurs réelles vs valeurs simplifiées

AVANT DE REpondre

Recherchez la datasheet d'une LED 5mm standard rouge ou verte.

Exemple : Vishay TLHR5400, Kingbright L-7113, ou tout équivalent.

Cherchez sur Google : '5mm LED 5mm datasheet PDF' ou 'LED rouge 5mm datasheet'.

Repérez les sections : Absolute Maximum Ratings et Electrical Characteristics.

? QUESTION DS-1

Dans la section 'Absolute Maximum Ratings' de la datasheet, trouvez et notez les valeurs suivantes. Puis expliquez pourquoi on ne doit JAMAIS dépasser ces valeurs, même brièvement (risques concrets).

Parametre	Valeur dans la datasheet	Unite
Courant direct max $I_F(\text{max})$		mA
Tension directe typique V_F (a $I_F = 20 \text{ mA}$)		V
Tension directe maximale $V_F(\text{max})$		V
Tension inverse max V_R		V
Puissance dissipée max P_D		mW
Longueur d'onde dominante (couleur)		nm
Angle de vision (viewing angle)		degres

Explication des risques si on dépasse ces valeurs (2 risques concrets minimum) :

? QUESTION DS-2

La tension de seuil V_F qu'on a utilisée dans nos calculs était de 2V. Quelle est la valeur REELLE indiquée dans la datasheet pour un courant de 20 mA ? Recalculez la résistance exacte avec cette vraie valeur de V_F . La valeur de 220 Ohm qu'on utilise est-elle suffisante pour protéger la LED ? Justifiez par le calcul.

V_F réel trouvé dans la datasheet : _____ V (pour $I_F = 20$ mA)

Recalcul de la résistance avec la vraie valeur de V_F :

? QUESTION DS-3

La datasheet mentionne un 'viewing angle' (angle de vision). Que signifie concrètement cette valeur ? Si vous placez la LED dans un boîtier avec une fenêtre de 5 mm de diamètre, l'angle de vision vous importe-t-il ? Dans quels produits professionnels faudrait-il choisir une LED avec un angle de vision très large (> 120 degrés) plutôt qu'un angle étroit (< 30 degrés) ?

6.2 Datasheet du microcontrôleur ATmega328P

RESSOURCE

Cherchez la datasheet officielle : 'ATmega328P datasheet Microchip'.
URL directe : microchip.com -> section AVR 8-bit Microcontrollers -> ATmega328P
Cherchez les sections : DC Characteristics et I/O Pin Characteristics.

? QUESTION DS-4

Dans la section 'DC Characteristics' de la datasheet ATmega328P : (a) Quel est le courant maximum I_{OH} (source) ou I_{OL} (sink) par pin I/O numérique ? (b) Quel est le courant total maximum que peut délivrer l'ensemble des pins VCC et GND de l'ATmega328P ? (c) Dans notre projet : calculez la consommation totale de nos 3 LEDs (15 mA chacune) et vérifiez si on respecte ces limites. Que risquerait-on si on branchait 10 LEDs sans résistances adaptées ?

Paramètre ATmega328P	Valeur dans la datasheet	Unité
Courant max par pin I/O en sortie I_{OH} (HIGH)		mA
Courant max par pin I/O en entrée I_{OL} (LOW)		mA

BAC PRO CIEL — TP ÉLECTRONIQUE : Gestionnaire de Mots de Passe		TP Électronique
Parametre ATmega328P	Valeur dans la datasheet	Unite
Courant total max sur tous les pins combines		mA
Tension VOH (sortie a l'etat HAUT, IOH = 5 mA)		V

Calcul de la consommation totale dans notre projet et verification de la limite :

? QUESTION DS-5

Toujours dans la datasheet ATmega328P : l'Arduino Uno fonctionne a 5V et 16 MHz. Cherchez le graphique 'Speed vs. Supply Voltage' ou 'Maximum Frequency'. (a) A quelle frequence max peut fonctionner l'ATmega328P si on l'alimente en 3.3V ? (b) Si on alimentait l'Arduino a 3.3V tout en laissant le quartz 16 MHz, que se passerait-il ? (c) Quel est le courant de consommation typique du microcontroleur en mode normal (Active Mode) a 5V et 16 MHz ?

6.3 Datasheet du module I2C — PCF8574

RESSOURCE

Le module I2C colle derriere le LCD utilise generalement un PCF8574 (NXP ou Texas Instruments). Cherchez : 'PCF8574 datasheet NXP PDF' ou directement sur ti.com ou nxp.com. C'est un 'Remote 8-Bit I/O Expander for I2C-Bus'.

? QUESTION DS-6

Dans la datasheet PCF8574, trouvez la section sur l'adressage I2C. (a) Convertissez l'adresse 0x27 en binaire sur 7 bits (montrez la conversion). (b) Le PCF8574 possede 3 broches d'adresse materielles A0, A1, A2. Combien d'adresses differentes peut-on configurer ? Combien de modules PCF8574 peut-on donc brancher sur le meme bus I2C ? (c) Quelle est la difference entre PCF8574 (adresses 0x20 a 0x27) et PCF8574A (adresses 0x38 a 0x3F) ? Pourquoi cette difference existe-t-elle ?

Conversion 0x27 en binaire : 0x27 = 2 = _____ 7 = _____ -> 0x27 = _____

? QUESTION DS-7

Cherchez dans la datasheet PCF8574 la vitesse maximale du bus I2C supportée. (a) Quelle est la fréquence maximale en mode 'Fast-mode' ? (b) L'Arduino utilise 100 kHz par défaut (`Wire.begin()`). Si on voulait accélérer à 400 kHz, comment modifier le code Arduino ? Cherchez la fonction `Wire.setClock()`. (c) Dans quel cas concret voudrait-on accélérer la vitesse I2C avec un écran LCD dans un projet embarqué ?

6.4 Analyse croisée et synthèse**? QUESTION DS-8**

En vous basant sur les datasheets consultées, imaginez qu'on alimente le projet avec 2 piles AA en série ($2 \times 1.5V = 3V$). Analysez composant par composant : (a) L'ATmega328P peut-il fonctionner à 3V ? À quelle fréquence max ? (b) La tension aux bornes des LEDs est-elle suffisante ? Recalculez la résistance si $VCC = 3V$ et $V_F = 2.1V$. (c) Le PCF8574 peut-il fonctionner à 3V ? Vérifiez dans sa datasheet la plage de tension VDD. (d) Proposez une solution technique pour faire fonctionner l'ensemble à 3V.

? QUESTION DS-9 — DEFI

Dans la datasheet ATmega328P, la tension V_{OH} (Voltage Output High) à l'état HAUT n'est pas exactement 5V : elle chute selon le courant tiré. Trouvez dans la datasheet la valeur de V_{OH} quand I_{OH} = 15 mA. Utilisez cette valeur réelle (et non 5V) pour recalculer la résistance de protection d'une LED. Comparez le courant résultant avec le calcul initial basé sur 5V. Conclusion : l'approximation 5V était-elle sécurisante (résistances sous-estimées ou surestimées) pour la protection de la LED ?

V_{OH} réel à I_{OH} = 15 mA trouve dans la datasheet : _____ V

Recalcul complet de la résistance avec V_{OH} réel :

SÉANCE 2 — MESURES, SCHÉMAS ET PUISSANCE

Pré-requis pour cette séance :

Avoir terminé et fait valider le câblage (Séance 1)
Circuit complet et fonctionnel sur Tinkercad

Objectifs de cette séance :

Dessiner les schémas normalisés IEC
Effectuer des mesures virtuelles et les analyser
Calculer puissance et autonomie sur batterie

SÉANCE 2 — DURÉE TOTALE : 3 HEURES

PARTIE 7 — SCHÉMAS ÉLECTRONIQUES NORMALISÉS IEC (~55 min)

POURQUOI DESSINER UN SCHÉMA NORMALISÉ ?

Un schéma de câblage (comme Tinkercad) montre à quoi ressemblent les composants physiquement.
Un schéma électronique normalisé utilise des SYMBOLES universels définis par la norme IEC 60617.

Avantages du schéma normalisé :

- Lisible par tout technicien dans le monde, quelle que soit la langue
- Indispensable pour la fabrication industrielle (PCB, documentation technique)
- Requis dans les dossiers professionnels, brevets, fiches SAV
- Plus clair et plus compact qu'un schéma de câblage

Dans votre vie professionnelle, vous lirez et produirez des schémas normalisés quotidiennement.

7.1 Symboles normalisés IEC — Tableau de référence

Apprenez ces symboles par coeur. Ils sont universels et vous les retrouverez dans tous vos TP et dans la vie professionnelle.

Composant	Symbole IEC (à reconnaître)	Caractéristique principale	Dans notre projet ?
Résistance	Rectangle avec valeur en Ohms (Ω)	Limiter le courant — $R = U/I$	Oui — $3 \times 220 \Omega$
LED	Diode avec 2 flèches (lumière émise)	Conduit dans un seul sens + émet lumière	Oui — 3 LEDs
Buzzer	Haut-parleur simplifié ou symbole buzzer	Convertit signal élec → son	Oui — 1 buzzer
Alimentation +5V	Ligne horizontale avec +5V	Source de tension continue	Oui — Arduino 5V
Masse (GND)	Ligne horizontale avec triangle pointant vers le bas ou 3 traits décroissants	Référence 0V	Oui — GND commun
Connexion de fils	Point noir à l'intersection de 2 fils	Jonction électrique — connexion active	Partout dans le circuit
Croisement sans connexion	Pont en arc ou croisement simple sans point	Les fils se croisent mais ne sont PAS connectés	A éviter — bien organiser
Microcontrôleur / CI	Rectangle avec broches numérotées	Circuit intégré multi-fonctions	Arduino Uno = ATmega328P

7.2 Exercice : Dessiner le schéma normalisé d'un circuit LED

Dessinez dans l'espace ci-dessous le schéma électronique NORMALISÉ d'une seule LED (LED verte, Pin 13 Arduino). Utilisez les symboles IEC corrects. Faites apparaître : la source de tension +5V, le pin de sortie Arduino (modélisé comme un interrupteur), la résistance 220 Ω avec sa valeur, la LED avec la bonne orientation (flèche lumière), la masse GND. Indiquez sur le schéma les valeurs de tension en chaque noeud.

[Espace de dessin — schéma normalisé IEC — 10 cm de hauteur]

Notez les tensions en chaque noeud : $V_{\text{Pin13}} = __\text{ V}$ | $V_{\text{anode LED}} = __\text{ V}$ | $V_{\text{cathode}} = __\text{ V}$

7.3 Exercice : Schéma du circuit complet — version simplifiée

Dessinez maintenant le schéma normalisé des 3 LEDs et du buzzer connectés à l'Arduino. Utilisez une représentation simplifiée : l'Arduino est représenté par un rectangle avec ses broches numérotées. Regroupez intelligemment les composants pour que le schéma soit lisible (sens de lecture : gauche $\rightarrow$ droite, haut $\rightarrow$ bas). Tous les GND se rejoignent sur un bus GND commun.

[Espace de dessin — 14 cm de hauteur — Schéma complet LEDs + Buzzer + Arduino]

? QUESTION 10

Quelle est la différence fondamentale entre un schéma de câblage (comme dans Tinkercad) et un schéma électronique normalisé ? Dans quel contexte professionnel utiliseriez-vous l'un ou l'autre ? Donnez un exemple concret pour chacun.

VALIDATION 4 — Schémas normalisés

Faites vérifier vos schémas par votre professeur :

Schéma LED simple : symboles corrects, valeurs indiquées, tensions notées

Schéma complet : lisible, bien organisé, GND commun visible

Signature du professeur : _____

PARTIE 8 — MESURES VIRTUELLES : MULTIMÈTRE ET OSCILLOSCOPE (~65 min)

MESURER, C'EST COMPRENDRE

En électronique professionnelle, on ne suppose pas — on MESURE.

Tinkercad intègre des instruments de mesure virtuels :

- Le MULTIMÈTRE : mesure tension (V), courant (A), résistance (Ω), continuité
- L'OSCILLOSCOPE : visualise l'évolution d'un signal électrique dans le temps

Ces outils sont indispensables pour : vérifier que le circuit fonctionne correctement, trouver une panne, valider ses calculs théoriques, analyser des signaux numériques.

Dans cette partie, vous allez effectuer des mesures réelles sur votre circuit Tinkercad et comparer les résultats avec vos calculs théoriques.

8.1 Le multimètre virtuel — Rappel d'utilisation

UTILISER LE MULTIMÈTRE DANS TINKERCAD

Pour ajouter un multimètre : composants → Instruments → Multimeter

Mesure de TENSION (voltmètre) :

- Sélectionnez le mode V (tension continue, symbole =)
- Branchez le fil rouge (+) sur le point à mesurer
- Branchez le fil noir (-) sur la masse (GND)
- IMPORTANT : le voltmètre se branche EN PARALLÈLE avec le composant à mesurer

Mesure de COURANT (ampèremètre) :

- Sélectionnez le mode A (courant continu)
- IMPORTANT : l'ampèremètre se branche EN SÉRIE dans le circuit
- Il faut couper le fil et insérer l'ampèremètre dans la coupure

Mesure de CONTINUITÉ :

- Mode son / continuité — bip si le circuit est fermé
- Utile pour vérifier qu'un fil est bien connecté

8.2 Mesures de tension — Tableau de relevés

Lancez la simulation sur Tinkercad. Utilisez le multimètre en mode tension (V) pour mesurer les tensions aux points indiqués. Comparez avec vos calculs théoriques et calculez l'écart.

Point de mesure	Valeur théorique calculée (V)	Valeur mesurée Tinkercad (V)	Écart (%) — Explication
Tension d'alimentation Arduino (5V)	5.00 V		
Tension aux bornes de la LED verte (Pin 13 HIGH)	~2.0 V		
Tension aux bornes de la résistance 220 Ω (LED verte)	~3.0 V		
Tension aux bornes de la LED verte (Pin 13 LOW)	0 V		

BAC PRO CIEL — TP ÉLECTRONIQUE : Gestionnaire de Mots de Passe			TP Électronique
Point de mesure	Valeur théorique calculée (V)	Valeur mesurée Tinkercad (V)	Écart (%) — Explication
Tension aux bornes de la LED bleue (Pin 12 HIGH)	~2.0 V		
Tension sur SDA (A4) pendant comm. I2C	Signal variable 0-5V		Observer les variations
Tension sur SCL (A5) pendant comm. I2C	Signal carré 0-5V		Observer les variations

8.3 Mesures de courant — Vérification expérimentale

Mesurez maintenant le courant qui traverse la LED verte. Pour cela, coupez le fil entre la résistance et l'anode de la LED, et insérez le multimètre en mode ampèremètre (A) dans cette coupure.

Mesure	Valeur théorique (mA)	Valeur mesurée (mA)	Cohérent ?
Courant dans la LED verte (Pin 13 HIGH)	~13.6 mA		☐ Oui ☐ Non
Courant dans la LED bleue (Pin 12 HIGH)	~13.6 mA		☐ Oui ☐ Non
Courant avec 2 LEDs allumées simultanément	~27.2 mA total		☐ Oui ☐ Non

? QUESTION 11

En observant le tableau de mesures : vos valeurs mesurées correspondent-elles à vos calculs théoriques ? Si vous observez un écart, expliquez pourquoi (modèle simplifié vs réalité, valeurs arrondies, tolérance des composants...). Que concluez-vous sur la fiabilité des calculs théoriques pour la conception d'un circuit ?

8.4 L'oscilloscope virtuel — Analyser les signaux

UTILISER L'OSCILLOSCOPE DANS TINKERCAD

L'oscilloscope affiche l'évolution de la tension au cours du temps.

Il permet de voir ce que le multimètre ne peut pas montrer : la forme des signaux.

Pour l'ajouter : composants → Instruments → Oscilloscope

Réglages de base :

- Time/div (axe horizontal) : ajustez pour voir 2 à 5 périodes du signal
- Volts/div (axe vertical) : ajustez pour que le signal occupe 60% de l'écran
- Channel A et Channel B : vous pouvez observer 2 signaux simultanément

Branchez la sonde Channel A sur le pin à observer, et le GND de la sonde sur GND Arduino.

8.5 Observation des signaux — Tableau d'analyse

Utilisez l'oscilloscope pour observer les signaux suivants. Pour chaque signal, dessinez la forme d'onde dans la case correspondante et notez les caractéristiques mesurées.

Signal observé	Forme d'onde (dessin)	Caractéristiques mesurées	Type de signal
Pin 13 — LED verte clignotante (1 Hz dans le code de test)	[Dessiner ici]	Fréquence : _____ Hz Période : _____ ms Tension haute : _____ V Tension basse : _____ V	Signal carré / numérique
Pin 10 — Buzzer : signal tone() envoyé (ex : tone(BUZZER, 1000))	[Dessiner ici]	Fréquence : _____ Hz Période : _____ µs Rapport cyclique : _____ %	Signal carré / PWM
Pin A5 — SCL (horloge I2C) pendant l'affichage sur le LCD	[Dessiner ici]	Fréquence estimée : _____ kHz Signal actif quand : _____ Signal inactif (repos) : _____ V	Horloge I2C — rafales

? QUESTION 12

Vous observez le signal SCL (horloge I2C) avec l'oscilloscope. Ce signal n'est pas continu : il apparaît par rafales puis s'arrête. Pourquoi ? Expliquez en lien avec le fonctionnement du protocole I2C. Quel est le lien entre la durée de ces rafales et la quantité de données transmises à l'écran LCD ?

? QUESTION 13

La fonction tone(BUZZER, 1000) envoie un signal à 1000 Hz. (a) Quelle est la période de ce signal en millisecondes ? Montrez le calcul. (b) Que se passe-t-il si vous appelez tone(BUZZER, 2000) — le son sera-t-il plus aigu ou plus grave ? Pourquoi ? (c) La fonction analogWrite() permet de générer un signal PWM (Pulse Width Modulation). En quoi le signal PWM est-il différent d'un simple signal carré 50/50 ? Cherchez la définition de 'rapport cyclique' (duty cycle).

PARTIE 9 — PUISSANCE, ÉNERGIE ET AUTONOMIE SUR BATTERIE (~55 min)

POURQUOI CALCULER LA CONSOMMATION ?

Tout système électronique embarqué doit être conçu en pensant à sa source d'énergie. Un smartphone, un objet connecté, un capteur industriel... tous ont une batterie limitée.

Savoir calculer la consommation d'un circuit permet de :

- Choisir la bonne batterie (capacité en mAh)
- Estimer l'autonomie du système
- Identifier les composants les plus gourmands en énergie
- Optimiser le circuit pour réduire la consommation (modes veille, etc.)

C'est une compétence clé en systèmes embarqués (IoT, domotique, industrie 4.0).

9.1 Rappels : Puissance et Énergie

FORMULES ESSENTIELLES

Puissance électrique (en Watts) :

$$P = U \times I \quad (\text{tension en V} \times \text{courant en A} = \text{puissance en W})$$

$$\text{Exemple : } P = 5V \times 0.015A = 0.075 W = 75 \text{ mW}$$

Énergie consommée (en Wh ou mWh) :

$$E = P \times t \quad (\text{puissance en W} \times \text{durée en heures} = \text{énergie en Wh})$$

$$\text{Exemple : } E = 75 \text{ mW} \times 2h = 150 \text{ mWh}$$

Autonomie d'une batterie :

$$\text{Autonomie (h)} = \text{Capacité batterie (mAh)} / \text{Courant consommé (mA)}$$

$$\text{Exemple : batterie 2000 mAh, consommation 100 mA} \rightarrow \text{autonomie} = 2000/100 = 20 \text{ heures}$$

Attention : on suppose ici que toutes les tensions sont les mêmes (5V).

En réalité, il faut tenir compte des rendements des convertisseurs de tension.

9.2 Budget de puissance du projet

Complétez le tableau de budget de puissance du projet gestionnaire de mots de passe. Rappel : courant moyen LED = 13.6 mA, buzzer actif = 30 mA, Arduino Uno actif = 50 mA, LCD I2C allumé = 25 mA, clavier (résistances pull-up internes) = 1 mA.

Composant	Tension (V)	Courant (mA)	Puissance (mW)	Remarque
Arduino Uno (microcontrôleur)	5	50		Toujours actif
Écran LCD I2C (allumé)	5	25		Rétroéclairage + chip
LED verte allumée	5	13.6		Résistance incluse
LED bleue allumée	5	13.6		Résistance incluse
LED jaune allumée	5	13.6		Résistance incluse
Buzzer actif (en fonctionnement)	5	30		Consomme si signal actif

BAC PRO CIEL — TP ÉLECTRONIQUE : Gestionnaire de Mots de Passe				TP Électronique
Composant	Tension (V)	Courant (mA)	Puissance (mW)	Remarque
Clavier matriciel (scan)	5	1		Résistances pull-up
TOTAL (tout allumé en même temps)	5			Scénario pire cas
TOTAL (utilisation normale — 1 LED + LCD + Arduino)	5			Scénario typique

9.3 Calculs d'autonomie — Scénarios

Calculez l'autonomie du système pour différentes sources d'énergie. Utilisez le courant de la colonne 'utilisation normale' du tableau ci-dessus.

Source d'énergie	Tension nominale	Capacité (mAh)	Autonomie calculée (h)	Autonomie en jours
Câble USB PC	5V	Illimitée (PC branché)	—	—
Power bank USB 5000 mAh	5V	5000		
Batterie Li-Ion 18650 (3.7V → 5V via régulateur)	5V (après conv.)	3000		
4× piles AA alcalines (6V → 5V via régulateur)	5V (après conv.)	2400		
Panneau solaire 1W + batterie 1000 mAh	5V	1000 (batterie seule)		Variable selon ensoleillement

? QUESTION 14

Le rétroéclairage du LCD consomme environ 20 mA à lui seul. Si on éteignait le rétroéclairage LCD quand l'écran n'affiche rien (mode veille après 10 secondes d'inactivité), combien de mA économiserait-on ? Recalculez l'autonomie avec la power bank 5000 mAh en mode normal VS mode veille activé. Exprimez le gain en heures et en pourcentage.

? QUESTION 15

Le mode veille de l'ATmega328P (Power-down sleep mode) réduit la consommation du microcontrôleur à environ 0.1 μ A (0.0001 mA). Si on pouvait mettre tout le système en veille profonde (Arduino + LCD éteint, LEDs éteintes, uniquement le clavier actif pour le réveil) avec une consommation totale de 0.5 mA, quelle serait l'autonomie avec une batterie 3000 mAh ? Ce mode veille est-il réaliste avec notre matériel actuel ? Quels composants supplémentaires faudrait-il ajouter ?

SÉANCE 3 — CODE, RAPPORT ET AMÉLIORATIONS

Pré-requis pour cette séance :

Séances 1 et 2 terminées et validées
Circuit câblé et testé sur Tinkercad

Objectifs de cette séance :

Lire, comprendre et annoter le code Arduino
Rédiger un compte-rendu technique professionnel
Proposer des améliorations matérielles réalistes

SÉANCE 3 — DURÉE TOTALE : 3 HEURES

PARTIE 10 — LECTURE ET ANNOTATION DU CODE ARDUINO (~65 min)

POURQUOI LIRE DU CODE SANS L'ÉCRIRE ?

Un technicien CIEL doit être capable de lire du code existant pour :

- Comprendre comment un système fonctionne (maintenance, SAV)
- Identifier la cause d'un bug ou d'un dysfonctionnement
- Modifier ou améliorer une fonctionnalité existante
- Câbler correctement le matériel en lisant les constantes de pin dans le code

Dans cette partie, vous allez analyser le code du gestionnaire de mots de passe sans l'écrire vous-même. Cela vous prépare à la partie code du prochain TP.

10.1 Structure d'un programme Arduino — Rappel

ARCHITECTURE DU CODE ARDUINO

Tout programme Arduino est structuré en 3 grandes zones :

ZONE 1 — En-tête (avant setup et loop) :

- `#include <bibliotheque.h>` : importe des fonctions déjà codées
- `const int PIN = 13;` : constantes (valeurs qui ne changent jamais)
- `int variable = 0;` : variables globales (accessibles partout)
- `struct NomStruct { ... };` : définition de structures de données

ZONE 2 — `setup()` — exécutée UNE SEULE FOIS au démarrage :

- `pinMode(pin, OUTPUT/INPUT)` : configure chaque pin en entrée ou sortie
- `lcd.begin()` : initialise les composants
- `Serial.begin(9600)` : active le moniteur série pour debug

ZONE 3 — `loop()` — exécutée EN BOUCLE INFINIE :

- Affichage du menu
- Lecture d'une touche du clavier
- Appel de la fonction correspondante

10.2 Analyse du code — Extrait à annoter

Lisez attentivement l'extrait de code ci-dessous. Pour chaque ligne numérotée, écrivez dans le tableau en dessous ce que fait cette ligne, en français simple et précis.

```

// — ZONE 1 : DÉCLARATIONS —————
L01: #include <Keypad.h>
L02: #include <Wire.h>
L03: #include <LiquidCrystal_I2C.h>

L04: LiquidCrystal_I2C lcd(0x27, 16, 2);

L05: const int LED_VERTE = 13;
L06: const int BUZZER     = 10;

L07: struct MotDePasse {
L08:     String site;
L09:     String mdpChiffre;
L10: };

L11: MotDePasse coffre[5];
L12: int nbMotsDePasse = 0;
L13: String cleSecrete = "SECRET2025";

// — ZONE 2 : SETUP —————
L14: void setup() {
L15:     lcd.init();
L16:     lcd.backlight();
L17:     pinMode(LED_VERTE, OUTPUT);
L18:     pinMode(BUZZER,    OUTPUT);
L19: }

// — CHIFFREMENT XOR —————
L20: void chiffrerXOR(char* msg, char* cle) {
L21:     int n = strlen(msg);
L22:     int k = strlen(cle);
L23:     for (int i = 0; i < n; i++) {
L24:         msg[i] = msg[i] ^ cle[i % k];
L25:     }
L26: }

```

Tableau d'annotation du code

Ligne	Extrait du code	Explication en français (ce que fait cette ligne)
L01	#include <Keypad.h>	
L02	#include <Wire.h>	
L03	#include <LiquidCrystal_I2C.h>	
L04	LiquidCrystal_I2C lcd(0x27,16,2)	
L05	const int LED_VERTE = 13	
L07-L10	struct MotDePasse { ... }	
L11	MotDePasse coffre[5]	
L12	int nbMotsDePasse = 0	
L13	String cleSecrete = "SECRET2025"	

Ligne	Extrait du code	Explication en français (ce que fait cette ligne)
L15	<code>lcd.init()</code>	
L17	<code>pinMode(LED_VERTE, OUTPUT)</code>	
L20	<code>void chiffrerXOR(char* msg, char* cle)</code>	
L21	<code>int n = strlen(msg)</code>	
L23	<code>for (int i=0; i<n; i++)</code>	
L24	<code>msg[i] = msg[i] ^ cle[i % k]</code>	

10.3 Questions de compréhension du code

? QUESTION 16

Ligne L13 : la clé secrète est stockée en clair dans le code : `String cleSecrete = 'SECRET2025'`. C'est une FAILLE DE SÉCURITÉ majeure. Expliquez pourquoi c'est dangereux. Comment un attaquant pourrait-il trouver cette clé ? Proposez 2 approches pour mieux protéger la clé secrète dans un vrai produit.

? QUESTION 17

Ligne L11 : `MotDePasse coffre[5]` — le tableau est limité à 5 mots de passe. (a) Quelle erreur se produira dans le code si l'utilisateur essaie d'en stocker un 6ème ? (b) Comment modifier la déclaration pour permettre 10 mots de passe ? (c) Quelle contrainte matérielle (mémoire) limite cette capacité sur un Arduino Uno ? Cherchez la quantité de RAM de l'ATmega328P dans sa datasheet.

? QUESTION 18

Tracez l'exécution de `chiffreXOR()` pas à pas pour chiffrer le mot 'CAT' avec la clé 'AB'. (a) Donnez la valeur ASCII (décimale) de C, A, T, A et B. (b) Calculez l'opération XOR pour chaque caractère : C XOR A = ?, A XOR B = ?, T XOR A = ? (c) Quels sont les codes ASCII des caractères chiffrés ? (d) Refaites l'opération avec le résultat chiffré et la même clé 'AB' pour vérifier qu'on retrouve bien 'CAT'. Montrez tous les calculs.

i	msg[i] (clair)	ASCII décimal	cle[i%k]	ASCII clé	msg[i] XOR cle = ?
0	C	67	A	65	
1	A	65	B	66	
2	T	84	A (i%2=0)	65	

Vérification du déchiffrement — XOR du résultat avec 'AB' :

PARTIE 11 — RÉDACTION DU COMPTE-RENDU TECHNIQUE (~55 min)**POURQUOI UN COMPTE-RENDU TECHNIQUE ?**

Dans votre futur métier, vous serez amenés à rédiger des documents techniques :

- Compte-rendu de TP ou d'intervention
- Rapport d'installation ou de mise en service
- Fiche de test et de recette (document validant qu'un système fonctionne)
- Rapport de panne et d'analyse

Ces documents sont lus par vos collègues, vos supérieurs, vos clients.

Ils doivent être clairs, précis, factuels et professionnels.

Exercice : rédigez le compte-rendu de ce TP selon le plan imposé ci-dessous.

11.1 Plan imposé du compte-rendu**STRUCTURE DU COMPTE-RENDU PROFESSIONNEL****1. PRÉSENTATION DU PROJET (5 à 8 lignes)**

Décrivez en quoi consiste le projet, son contexte professionnel, son utilité.

2. LISTE DU MATÉRIEL UTILISÉ (tableau)

Référence, quantité, rôle dans le projet.

3. DESCRIPTION DES CONNEXIONS (5 à 10 lignes + tableau récapitulatif)

Expliquez comment vous avez câblé les composants, en justifiant les choix.

4. RÉSULTATS DES TESTS (tableau)

Pour chaque composant : test effectué, résultat attendu, résultat obtenu.

5. DIFFICULTÉS RENCONTRÉES (5 lignes minimum)

Décrivez un problème que vous avez rencontré et comment vous l'avez résolu.

Si aucun problème : décrivez le problème que vous auriez pu rencontrer.

6. ANALYSE CRITIQUE ET AMÉLIORATIONS (6 à 10 lignes)

Que fonctionnerait-il différemment dans un vrai produit industriel ?

Quelles améliorations proposeriez-vous ?

7. CONCLUSION (3 à 5 lignes)

Bilan personnel : qu'avez-vous appris ? Qu'est-ce qui vous a le plus intéressé ?

1. Présentation du projet

BAC PRO CIEL — TP ÉLECTRONIQUE : Gestionnaire de Mots de Passe			TP Électronique
Composant testé	Test effectué	Résultat attendu	Résultat obtenu

5. Difficultés rencontrées et solutions apportées

6. Analyse critique et améliorations proposées

7. Conclusion personnelle

PARTIE 12 — MINI CAHIER DES CHARGES : AMÉLIORATIONS MATÉRIELLES (~55 min)

LA DÉMARCHE D'INGÉNIERIE

Un produit électronique n'est jamais 'fini' : il peut toujours être amélioré.

Dans l'industrie, on rédige un 'cahier des charges' (CDC) avant de modifier un produit.

Le CDC décrit : ce que le système doit faire (besoins), comment il doit le faire (contraintes), et ce qui ne doit pas changer (critères d'acceptation).

Dans cette partie, vous allez jouer le rôle d'ingénieur et proposer des améliorations concrètes au gestionnaire de mots de passe, avec justifications techniques.

12.1 Analyse des limites du système actuel

Avant de proposer des améliorations, identifiez les limites du système tel qu'il est :

Aspect du système	Limite identifiée	Impact pour l'utilisateur
Stockage des mots de passe	Mémoire RAM volatile : tout est perdu à l'extinction	Doit re-saisir tous les mots de passe à chaque démarrage
Capacité maximale	Maximum 5 mots de passe (tableau coffre[5])	
Sécurité du chiffrement	Algorithme XOR facilement cassable	
Interface utilisateur	Clavier numérique — saisie lente (système T9)	
Portabilité / Alimentation	Nécessite un câble USB — pas autonome	
Protection physique	Aucun boîtier — composants exposés	
Authentification	Aucune protection contre les accès non autorisés au démarrage	

12.2 Fiches d'amélioration — Choisissez-en 2 à développer

Pour chaque amélioration choisie, remplissez complètement la fiche ci-dessous. Soyez précis et réaliste dans vos propositions matérielles.

AMÉLIORATION DISPONIBLES — Choisissez 2 parmi :

- A. Sauvegarde permanente via module EEPROM externe (AT24C256 — 256 Ko)
- B. Affichage sur écran OLED 128×64 pixels à la place du LCD 16×2
- C. Authentification biométrique par module lecteur d'empreintes digitales
- D. Communication sans fil Bluetooth via module HC-05 pour synchronisation PC
- E. Alimentation autonome via batterie Li-Po rechargeable + circuit de charge TP4056
- F. Clavier QWERTY miniature I2C pour saisie rapide du texte
- G. Buzzer piézoélectrique passif à la place du buzzer actif (pour mélodies d'alerte)
- H. LED RGB unique à la place des 3 LEDs séparées (économie de pins)

FICHE D'AMÉLIORATION N°1 — Amélioration choisie : _____ (lettre)

BAC PRO CIEL — TP ÉLECTRONIQUE : Gestionnaire de Mots de Passe		TP Électronique
Intitulé de l'amélioration		
Problème résolu		
Nouveau composant nécessaire (référence)		
Pins Arduino utilisés (lesquels ?)		
Protocole / interface utilisé (I2C, SPI, UART, numérique...)		
Coût estimé du composant		
Schéma de connexion (croquis)	<i>[Dessiner le schéma de connexion du nouveau composant]</i>	
Bénéfice pour l'utilisateur final		
Difficulté / risque de mise en oeuvre		

FICHE D'AMÉLIORATION N°2 — Amélioration choisie : _____ (lettre)

Intitulé de l'amélioration		
Problème résolu		
Nouveau composant nécessaire (référence)		
Pins Arduino utilisés (lesquels ?)		
Protocole / interface utilisé (I2C, SPI, UART...)		
Coût estimé du composant		<i>[Schéma de connexion]</i>
Bénéfice pour l'utilisateur final		
Difficulté / risque de mise en oeuvre		

? QUESTION 19

Si vous deviez choisir UNE SEULE amélioration à implémenter en priorité pour rendre ce produit commercialisable, laquelle choisiriez-vous et pourquoi ? Justifiez votre choix en termes de sécurité, d'ergonomie et de faisabilité technique. Quel serait le coût total estimé du projet amélioré ?

SÉANCES 4 & 5 — ÉCRIRE LE CODE ARDUINO

SÉANCES 4 & 5 — DURÉE TOTALE : 5 HEURES

PARTIE 13 — LES BASES DU LANGAGE C/C++ ARDUINO (~60 min)

POURQUOI APPRENDRE À CODER ?

Le câblage électronique c'est le corps du système. Le code, c'est le cerveau.

Sans code, les composants ne font rien. C'est le code qui leur donne un comportement.

Arduino utilise un sous-ensemble du C++ avec des fonctions spécifiques aux microcontrôleurs.

La bonne nouvelle : les concepts de base s'apprennent vite et sont réutilisables dans tous les langages (Python, Java, JavaScript...). Apprendre C sur Arduino, c'est apprendre à penser comme une machine.

13.1 Les types de données — Tableau de référence

En C/C++ chaque variable a un TYPE qui détermine la taille en mémoire et les valeurs possibles. C'est crucial sur un Arduino Uno qui n'a que 2 Ko de RAM !

Type	Taille	Plage de valeurs	Exemple d'usage	Dans notre projet ?
bool	1 octet	true / false (0 ou 1)	bool led_allumee = false;	État des LEDs
byte	1 octet	0 à 255	byte adresse = 0x27;	Adresse I2C LCD
int	2 octets	-32 768 à 32 767	int compteur = 0;	nbMotsDePasse
unsigned int	2 octets	0 à 65 535	unsigned int duree = 5000;	delay() en ms
long	4 octets	-2.1 M à 2.1 M	long tempo = millis();	Compteur de temps
float	4 octets	$\pm 3.4 \times 10^{38}$ (6 décimales)	float tension = 4.87;	Mesure tension
char	1 octet	-128 à 127 ou 1 caractère	char lettre = 'A';	Caractères saisis
char[]	n octets	Chaîne de n-1 caractères	char mdp[50] = "secret";	Mots de passe
String	Variable	Texte de longueur variable	String site = "GMAIL";	Noms des sites

ATTENTION — String vs char[] sur Arduino

La classe String (majuscule) est commode mais fragmente la mémoire RAM sur Arduino.

Pour des programmes robustes en production, on préfère char[] (tableau de caractères).

Dans ce projet pédagogique, on utilise String pour simplifier — mais c'est à éviter en pro.

Règle : si vous voyez des plantages aléatoires sur Arduino → remplacez String par char[].

? QUESTION C1

Calculez la mémoire totale occupée par le tableau coffre[5] sachant que chaque MotDePasse contient 2 String de 20 caractères en moyenne. Rappel : un char = 1 octet, une String a aussi un overhead de ~24

octets. L'Arduino Uno a 2048 octets de RAM. Quel pourcentage de la RAM est utilisé uniquement pour ce tableau ? Que se passe-t-il si on augmente coffre[5] à coffre[50] ?

13.2 Les structures de contrôle — Tableaux d'exercices

LES 4 STRUCTURES DE CONTRÔLE FONDAMENTALES

1. if / else if / else : exécution conditionnelle
2. for : boucle avec compteur
3. while : boucle tant que condition vraie
4. switch / case : choix multiple (menu)

Ces 4 structures permettent d'écrire N'IMPORTE QUEL algorithme.

Un programme complexe n'est qu'une combinaison de ces 4 structures.

Exercice 1 — Lire du code et prédire le résultat

Pour chaque extrait, écrivez ce que va afficher le Serial Monitor sans exécuter le code. Justifiez.

Code Arduino	Que s'affiche-t-il ? (justifiez)
<pre>int x = 10; if (x > 5) { Serial.println("Grand"); } else { Serial.println("Petit"); }</pre>	
<pre>for (int i=0; i<4; i++) { Serial.println(i * 2); }</pre>	
<pre>int n = 3; switch (n) { case 1: Serial.println("Un"); break; case 2: Serial.println("Deux"); break; case 3: Serial.println("Trois"); break; default: Serial.println("Autre"); }</pre>	

Exercice 2 — Corriger les bugs

Chaque extrait contient 1 ou 2 erreurs. Trouvez-les, expliquez le problème et réécrivez le code corrigé.

Code avec bug(s)	Erreur(s) identifiée(s)	Code corrigé
<pre>int score = 0 if (score = 100) { Serial.println("Parfait!"); } void allumerLED() { int pin = 13 digitalWrite(pin, HIGH); delay(1000) digitalWrite(pin, LOW) char mdp[5]; strcpy(mdp, "FACEBOOK"); Serial.println(mdp);</pre>		

13.3 Les fonctions — Conception et écriture

ANATOMIE D'UNE FONCTION EN C/C++

```
type_retour nomFonction(type param1, type param2) {
    // corps de la fonction
    return valeur; // si type_retour != void
}
```

type_retour : void (ne retourne rien), int, bool, char*, String...

nomFonction : en camelCase, verbe + complément (ex: lireTouche, chiffrerXOR)

paramètres : variables reçues en entrée — peuvent être modifiées si pointeurs (char*)

return : valeur renvoyée à l'appelant (obligatoire si type_retour != void)

Une bonne fonction fait UNE SEULE chose et fait-la bien.

Exercice 3 — Compléter des fonctions à trous

Chaque fonction ci-dessous a des trous (____). Complétez-les pour que la fonction fasse ce que son commentaire décrit.

```
// FONCTION 1 — Faire clignoter une LED n fois avec une pause de ms millisecondes
void clignoteN(int pin, int n, int ms) {
    for (int i = 0; i < ____; i++) {
        digitalWrite(____, HIGH);
        delay(____);
        digitalWrite(____, LOW);
        delay(____);
    }
}

// FONCTION 2 — Retourner true si la String 's' est vide ou fait moins de 3 caractères
____ estTropCourt(String s) {
    return (s.____ < ____);
}

// FONCTION 3 — Chercher un site dans coffre[] et retourner son index (-1 si non trouvé)
int trouverSite(String nomSite) {
    for (int i = 0; i < ____; i++) {
        if (coffre[i].____ == ____) {
            return ____;
        }
    }
}
```

```
    }  
}  
return ____; // non trouvé  
}
```

PARTIE 14 — ÉCRIRE LES FONCTIONS PRINCIPALES DU PROJET (~80 min)

MÉTHODE : PENSER AVANT D'ÉCRIRE

Avant d'écrire une seule ligne de code, un bon développeur :

1. Décrit en français ce que la fonction doit faire (algorithme en langage naturel)
2. Identifie les entrées (paramètres) et la sortie (type de retour)
3. Liste les cas particuliers à gérer (erreurs, limites)
4. PUIS écrit le code

Ce processus s'appelle la conception algorithmique. C'est la compétence la plus importante — le code, c'est juste la traduction de votre raisonnement.

14.1 Algorithme en français — Exercice préparatoire

Avant d'écrire chaque fonction, rédigez son algorithme en français dans le tableau. Soyez précis et numérotez les étapes.

Fonction à écrire	Algorithme en français (étapes numérotées)	Cas particuliers à gérer
supprimerMotDePasse(String site) <i>Supprime le mot de passe associé au site donné du tableau coffre[]</i>		
afficherListe() <i>Affiche la liste de tous les sites stockés sur le LCD (défilement si > 2 sites)</i>		
verifierCle(String saisie) <i>Retourne true si la clé saisie correspond à cleSecrete, false sinon. Incrémente un compteur d'échecs.</i>		

14.2 Écrire la fonction supprimerMotDePasse()

Écrivez le code complet de la fonction `supprimerMotDePasse(String site)` dans l'espace ci-dessous. Utilisez votre algorithme de la section 14.1. Indices : utilisez `trouverSite()` pour trouver l'index, puis décalez les éléments du tableau vers la gauche pour combler le trou, puis décrementez `nbMotsDePasse`.

```
void supprimerMotDePasse(String site) {
    // Étape 1 : chercher l'index du site
    int idx = _____;

    // Étape 2 : si non trouvé, afficher erreur et sortir
    if (idx == _____) {
        lcd.clear();
        lcd.print(_____);
        delay(2000); return;
    }
}
```

```

    }

    // Étape 3 : décaler tous les éléments après idx vers la gauche
    for (int i = ____; i < ____; i++) {
        coffre[i] = coffre[____];
    }

    // Étape 4 : décrémenter le compteur
    ____;

    // Étape 5 : confirmation visuelle
    lcd.clear();
    lcd.print(____);
    digitalWrite(LED_JAUNE, ____);
    delay(2000);
    digitalWrite(LED_JAUNE, ____);
}

```

14.3 Écrire la fonction verifierCle() complète

Écrivez cette fonction EN ENTIER dans l'espace libre ci-dessous. Elle doit : comparer la saisie à cleSecrete, incrémenter un compteur global echecs si la clé est fausse, bloquer le système (while(1){}) après 3 échecs consécutifs avec une alarme buzzer, et remettre le compteur à 0 si la clé est correcte.

```

int echecs = 0;    // variable globale – à déclarer en haut du programme

bool verifierCle(String saisie) {

}

```

? QUESTION C2

La fonction verifierCle() utilise une comparaison directe de chaînes de caractères (== ou .equals()). En cybersécurité, on parle d'attaque 'timing attack' : si la comparaison s'arrête dès le premier caractère différent, un attaquant peut mesurer le temps de réponse pour deviner la clé caractère par caractère. Comment écrire une comparaison en temps constant qui évite cette fuite d'information ? Cherchez 'constant-time comparison' et proposez une implémentation en C.

PARTIE 15 — DÉBOGAGE ET SERIAL MONITOR (~55 min)

LE DÉBOGAGE : UNE COMPÉTENCE PROFESSIONNELLE CLEF

80% du temps d'un développeur est consacré au débogage (trouver et corriger des bugs).

Le Serial Monitor d'Arduino est votre outil principal de débogage.

Il permet d'afficher la valeur de n'importe quelle variable à n'importe quel moment.

Règle d'or du débogage : ISOLER → REPRODUIRE → IDENTIFIER → CORRIGER → VÉRIFIER

Ne jamais corriger 'au hasard'. Chaque modification doit être justifiée par une hypothèse.

15.1 Instrumenter le code — Ajouter des traces de debug

Voici la fonction `stockerMotDePasse()` sans aucune trace de debug. Ajoutez dans les espaces prévus les `Serial.println()` qui vous permettront de suivre l'exécution pas à pas. Indiquez quelle valeur vous affichez et pourquoi.

```
void stockerMotDePasse() {
    // [DEBUG 1] _____

    if (nbMotsDePasse >= 5) {
        lcd.print("MEMOIRE PLEINE!"); delay(2000); return;
    }
    lcd.print("Nom du site:"); String site = saisirTexte();
    // [DEBUG 2] _____

    if (site == "") return;
    lcd.print("Mot de passe:"); String mdp = saisirTexte();
    // [DEBUG 3] _____

    char buf[50]; strcpy(buf, mdp.c_str());
    char cle[50]; strcpy(cle, cleSecrete.c_str());
    chiffreurXOR(buf, cle);
    // [DEBUG 4] _____

    coffre[nbMotsDePasse].site = site;
    coffre[nbMotsDePasse].mdpChiffre = String(buf);
    nbMotsDePasse++;
    // [DEBUG 5] _____
    digitalWrite(LED_VERTE, HIGH); delay(2000); digitalWrite(LED_VERTE, LOW);
}
```

15.2 Scénarios de bugs — Diagnostic

Pour chaque symptôme, identifiez le bug probable et proposez un test de diagnostic avec le Serial Monitor.

Symptôme observé	Bug probable (hypothèse précise)	Test Serial Monitor pour confirmer
Le mot de passe récupéré est différent de celui qui avait été stocké		
Le système se bloque après avoir stocké 3 mots de passe (jamais le 4ème)		

BAC PRO CIEL — TP ÉLECTRONIQUE : Gestionnaire de Mots de Passe		TP Électronique
Symptôme observé	Bug probable (hypothèse précise)	Test Serial Monitor pour confirmer
La LED verte s'allume mais le mot de passe n'est pas sauvegardé — liste vide L'Arduino redémarre tout seul après 30 secondes d'utilisation intensive		

SÉANCES 6 & 7 — DURÉE TOTALE : 6 HEURES

PARTIE 16 — REPRÉSENTATION DES DONNÉES : BINAIRE, HEX, ASCII (~60 min)

TOUT EST DES NOMBRES

Un ordinateur ne comprend que des 0 et des 1 (bits). Tout — texte, images, mots de passe — est stocké sous forme de nombres binaires. Comprendre cette représentation est fondamental en cybersécurité : chiffrer, c'est transformer des nombres par des opérations mathématiques.

3 systèmes de numération à maîtriser :

- Binaire (base 2) : 0 et 1 uniquement — langage natif des circuits
- Hexadécimal (base 16) : 0-9 puis A-F — compact, lisible par les humains
- Décimal (base 10) : notre système habituel — pour les humains

16.1 Conversions — Tableaux à compléter

Complétez entièrement ce tableau. Montrez vos calculs dans l'espace en dessous.

Décimal	Binaire (8 bits)	Hexadécimal	Caractère ASCII
65	0100 0001	0x41	A
66		0x42	
	0100 0011		C
97			
	0110 0001		
48		0x21	
127			
32			espace (SP)

Espace de calcul :

16.2 L'opération XOR bit à bit — Exercices complets

RÈGLE DU XOR

XOR ($\oplus$) : le résultat est 1 si les bits sont DIFFÉRENTS, 0 s'ils sont IDENTIQUES

$$0 \oplus 0 = 0 \quad 0 \oplus 1 = 1 \quad 1 \oplus 0 = 1 \quad 1 \oplus 1 = 0$$

Propriété magique : $A \oplus B \oplus B = A$ (le XOR est sa propre inverse !)

Preuve : $(A \oplus B) \oplus B = A \oplus (B \oplus B) = A \oplus 0 = A$

C'est exactement pour ça que la même fonction chiffre ET déchiffre !

Valeur A (binaire)	Valeur B (binaire)	A XOR B (binaire)	Résultat décimal	Résultat hex
0100 0001 (A=65)	0100 0010 (B=66)			
0100 1101 (M=77)	0101 1000 (X=88)			
0110 0001 (a=97)	0100 0001 (A=65)			
0101 0011 (S=83)	0101 0011 (S=83)			

? QUESTION CY1

La 4ème ligne du tableau (S XOR S) donne un résultat intéressant. Que se passe-t-il quand on fait XOR d'un nombre avec lui-même ? Quel problème de sécurité cela crée-t-il si deux mots de passe différents sont chiffrés avec la MÊME clé XOR ? (Indice : calculez MDP1_chiffré XOR MDP2_chiffré et voyez ce que ça donne.)

PARTIE 17 — ATTAQUES RÉELLES SUR LE CHIFFREMENT XOR (~80 min)

APPRENDRE À ATTAQUER POUR MIEUX DÉFENDRE

En cybersécurité, comprendre comment un système peut être attaqué est indispensable pour le concevoir de manière sécurisée. On ne peut pas défendre ce qu'on ne comprend pas.

Dans cette partie, vous allez VOUS-MÊMES casser des textes chiffrés avec XOR.

Vous comprendrez ainsi pourquoi XOR n'est pas un chiffrement sécurisé.

Ces techniques — analyse de fréquence, known-plaintext — sont utilisées par de vrais cryptanalystes dans des contextes professionnels (renseignement, forensics, pentest).

17.1 Attaque 1 — Clé de longueur 1 (le plus simple)

Si la clé ne fait qu'un seul octet, chaque caractère du texte chiffré est XORé avec la MÊME valeur. On peut casser cela par force brute : il suffit de tester les 256 valeurs possibles et de choisir celle qui donne un texte lisible.

TEXTE CHIFFRÉ À DÉCODER (clé = 1 octet inconnu)

Texte chiffré en hexadécimal :

25 40 27 27 32 31 40 41 27 36

Ce texte est le résultat de : message_clair XOR 0x4F (pour chaque caractère)

Votre mission : décoder ce message en calculant $\text{chiffre}[i] \text{ XOR } 0x4F$ pour chaque octet.

Rappel : $0x25 \text{ XOR } 0x4F = ?$

$0x25 = 0010\ 0101$

$0x4F = 0100\ 1111$

$\text{XOR} = 0110\ 1010 = 0x6A = 106 \text{ décimal} = \text{lettre 'j' en ASCII}$

Position	Valeur chiffrée (hex)	Valeur chiffrée (déc)	Clé 0x4F (déc = 79)	Résultat décimal (XOR)	Caractère ASCII
0	0x25	37	79	37 XOR 79 = ?	
1	0x40	64	79		
2	0x27	39	79		
3	0x27	39	79		
4	0x32	50	79		
5	0x31	49	79		
6	0x40	64	79		
7	0x41	65	79		
8	0x27	39	79		
9	0x36	54	79		

Message déchiffré : _ _ _ _ _

17.2 Attaque 2 — Known-Plaintext Attack (KPA)

L'attaque par texte connu est redoutable : si l'attaquant sait qu'une partie du message contient un mot connu, il peut calculer des morceaux de la clé.

SCÉNARIO

Un attaquant a intercepté un texte chiffré XOR. Il sait que tous les mots de passe stockés commencent par le nom du site en majuscules. Il intercepte un coffre XOR avec la clé et il sait que le 1er site stocké est 'GMAIL'.

Texte chiffré (hex) des 5 premiers octets : 05 2E 28 24 2D

Texte clair connu : G M A I L (ASCII : 71 77 65 73 76)

Formule : $\text{clé}[i] = \text{chiffré}[i] \text{ XOR } \text{clair}[i]$

i	chiffré[i] hex (déc)	clair[i] char (déc)	clé[i] = chiffré XOR clair	Caractère de la clé
0	0x05 (5)	G (71)	5 XOR 71 = ?	
1	0x2E (46)	M (77)		
2	0x28 (40)	A (65)		
3	0x24 (36)	I (73)		
4	0x2D (45)	L (76)		

Clé partielle trouvée : _ _ _ _ _ (5 premiers caractères)

? QUESTION CY2

Avec ces 5 caractères de clé retrouvés, expliquez comment l'attaquant peut : (a) vérifier si la clé se répète (en cherchant si les 5 caractères forment un motif cohérent) ; (b) utiliser ces 5 caractères pour essayer de déchiffrer d'autres mots de passe stockés dans le coffre ; (c) étendre sa connaissance de la clé en ciblant d'autres mots connus (les noms de sites sont souvent prévisibles : GMAIL, FACEBOOK, AMAZON...). Conclure : pourquoi un attaquant qui connaît seulement le NOM des sites peut-il casser tout le coffre ?

17.3 Attaque 3 — Analyse de fréquence

PRINCIPE

En français, certaines lettres apparaissent beaucoup plus souvent que d'autres :

E : 17% | A : 8% | S : 7% | I : 7% | T : 7% | N : 7%

Si on chiffre un long texte avec XOR et une clé courte, les patterns de fréquence du texte clair 'transparaissent' dans le texte chiffré.

Méthode : compter les octets les plus fréquents dans le texte chiffré, supposer qu'ils correspondent à 'E' (0x45) ou espace (0x20), calculer la clé probable, et tester sur le texte complet.

? QUESTION CY3

Voici un texte chiffré (extrait) et l'analyse de fréquence des octets. L'octet 0x25 apparaît 18 fois sur 100. En supposant qu'il correspond au 'E' (0x45) : calculez le caractère de clé probable ($0x25 \text{ XOR } 0x45 = ?$). Ensuite, supposez qu'il correspond à un espace (0x20) : calculez ce que donne $0x25 \text{ XOR } 0x20$. Lequel donne un caractère de clé plus 'probable' pour un mot de passe humain ? Justifiez.

PARTIE 18 — CRYPTOGRAPHIE MODERNE : HACHAGE ET PROTECTION DES MOTS DE PASSE (~75 min)

CE QUE FONT VRAIMENT LES VRAIS GESTIONNAIRES DE MOTS DE PASSE

LastPass, Bitwarden, KeePass ne stockent PAS vos mots de passe chiffrés avec XOR. Ils utilisent une architecture cryptographique sophistiquée :

1. Votre mot de passe maître est passé dans une fonction de HACHAGE (SHA-256 + PBKDF2)
2. Le résultat (hash) devient la clé de chiffrement AES-256
3. Vos mots de passe sont chiffrés avec AES-256 dans un mode sécurisé (GCM)
4. La clé (votre hash) n'est JAMAIS stockée — elle est recalculée à chaque connexion

Résultat : même si la base de données est volée, elle est inexploitable sans votre MDP maître.

18.1 Le hachage cryptographique — Concepts

QU'EST-CE QU'UNE FONCTION DE HACHAGE ?

Une fonction de hachage transforme une entrée de taille quelconque en une sortie de taille fixe (le 'hash' ou 'empreinte').

Propriétés fondamentales :

- Déterministe : la même entrée donne toujours le même hash
- Sens unique (one-way) : impossible de retrouver l'entrée depuis le hash
- Résistante aux collisions : impossible de trouver 2 entrées différentes avec même hash
- Effet avalanche : 1 bit changé → hash complètement différent

Exemples : SHA-256 produit un hash de 256 bits (64 caractères hex) toujours.

hash('bonjour') = a7305e36... (64 caractères hex)

hash('Bonjour') = f344d4a9... (64 caractères hex — complètement différent !)

hash('bonjour') = a7305e36... (toujours le même — déterministe)

18.2 Exercice pratique — Analyser des hashes

Les hashes SHA-256 ci-dessous ont été calculés à partir de mots de passe courants. Utilisez un outil en ligne (ex : crackstation.net, hashcat, ou un simple dictionnaire) pour tenter de retrouver les mots de passe originaux.

OBJECTIF PÉDAGOGIQUE

Cet exercice montre que hacher directement un mot de passe sans sel (salt) est insuffisant.

Les 'rainbow tables' précalculent des millions de hashes — crackstation en contient 15 milliards.

C'est pour ça que les systèmes modernes ajoutent un 'sel' aléatoire avant de hacher.

Hash SHA-256	Mot de passe retrouvé	Technique utilisée
5e884898da28047151d0e56f8dc6292773603d0d6aabbdd62a11ef721d1542d8		
e10adc3949ba59abbe56e057f20f883e		(MD5 — plus court)
8d969eef6ecad3c29a3a629280e686cf0c3f5d5a86aff3ca12020c923adc6c92		

BAC PRO CIEL — TP ÉLECTRONIQUE : Gestionnaire de Mots de Passe		TP Électronique
Hash SHA-256	Mot de passe retrouvé	Technique utilisée
b14a7b8059d9c055954c92674ce60032		(MD5)

? QUESTION CY4

Sur les hashes que vous avez réussi à casser : (a) combien de temps a pris le cassage ? (b) Pourquoi ces mots de passe étaient-ils si faciles à retrouver malgré l'utilisation de SHA-256 (considéré comme robuste) ? (c) Quel type de mots de passe résisterait davantage aux rainbow tables ? Proposez 3 règles concrètes pour créer un mot de passe résistant au cassage par dictionnaire.

18.3 Le sel (salt) — Défense contre les rainbow tables

COMMENT FONCTIONNE LE SEL ?

Un 'sel' est une chaîne aléatoire ajoutée au mot de passe AVANT de le hacher.

Sans sel :

hash('motdepasse') = 5e884898... → dans les rainbow tables !

Avec sel (sel aléatoire = 'xK9#mL2p') :

hash('xK9#mL2p' + 'motdepasse') = 7f3a91b2... → PAS dans les rainbow tables !

Le sel est stocké en CLAIR à côté du hash — ce n'est pas un secret.

Son rôle est de rendre chaque hash UNIQUE même pour le même mot de passe.

Algorithmes modernes de hachage avec sel intégré : bcrypt, scrypt, Argon2

Argon2 est le gagnant du Password Hashing Competition (PHC) de 2015 — le standard actuel.

? QUESTION CY5

Voici 2 lignes d'une base de données de mots de passe hachés avec sel : Utilisateur 1 : sel = 'aB3#kL', hash = sha256('aB3#kL' + mot_de_passe_1) Utilisateur 2 : sel = 'zX7!mN', hash = sha256('zX7!mN' + mot_de_passe_1) (Les deux ont le même mot de passe !) (a) Ces deux hashes sont-ils identiques ? Pourquoi ? (b) Si un attaquant vole la base et sait que l'utilisateur 1 a le mot de passe 'azerty', peut-il directement savoir que l'utilisateur 2 a aussi 'azerty' ? (c) Combien d'entrées de rainbow table faudrait-il précalculer pour couvrir tous les sels possibles de 6 caractères (minuscules + chiffres + 3 symboles = 39 caractères) ? Calculez 39^6 .

PARTIE 19 — AES ET ARCHITECTURE D'UN VRAI GESTIONNAIRE (~75 min)

AES — ADVANCED ENCRYPTION STANDARD

AES est le standard mondial de chiffrement symétrique depuis 2001 (NIST).

Il est utilisé partout : HTTPS, WiFi WPA2/3, disques durs chiffrés, VPN, Signal, WhatsApp...

Caractéristiques :

- Clé de 128, 192 ou 256 bits (AES-128, AES-192, AES-256)
- Chiffrement par blocs de 128 bits (16 octets)
- 10, 12 ou 14 tours de transformations (selon la taille de clé)
- Approuvé par la NSA pour les documents SECRET (AES-256)

Contrairement à XOR :

- Non linéaire (résistant à l'analyse de fréquence)
- Diffusion complète (1 bit changé → tous les octets du bloc changent)
- Aucune attaque connue praticable avec la puissance de calcul actuelle

19.1 Les modes d'opération AES — Comprendre les différences

AES chiffre des blocs de 16 octets. Pour chiffrer des messages plus longs, on utilise des 'modes d'opération'. Certains sont sécurisés, d'autres non.

Mode	Principe simplifié	Problème de sécurité	Utilisé pour
ECB	Chaque bloc chiffré indépendamment	DANGEREUX : blocs identiques → chiffrés identiques. Les patterns restent visibles.	Ne JAMAIS utiliser en prod
CBC	Chaque bloc XORé avec le précédent avant chiffrement	Nécessite un IV (vecteur d'initialisation) aléatoire. Pas d'authentification intégrée.	Chiffrement de fichiers (avec HMAC en plus)
CTR	AES utilisé comme générateur de flux pseudo-aléatoire	Nécessite un nonce unique — réutiliser un nonce est catastrophique	Chiffrement rapide, parallélisable
GCM	CTR + authentification intégrée (GHASH)	Le mode recommandé. Vérifie intégrité ET confidentialité	HTTPS, messageries, gestionnaires de MDP

? QUESTION CY6

Le mode ECB est dangereux à cause des 'patterns visibles'. Cherchez sur internet 'AES ECB mode penguin' (l'image du manchot). Expliquez pourquoi une image chiffrée avec AES-ECB révèle encore la forme du manchot original, alors qu'AES est censé être un chiffrement robuste. Quel est le principe fondamental de sécurité que ECB viole (cherchez 'indistinguishability') ?

19.2 Architecture complète d'un gestionnaire sécurisé

Comparez l'architecture du projet Arduino (chiffrement XOR) avec un gestionnaire professionnel (Bitwarden open-source). Pour chaque fonctionnalité de sécurité, notez si le projet Arduino la possède ou non.

Fonctionnalité de sécurité	Bitwarden (vrai gestionnaire)	Notre projet Arduino
Algorithme de chiffrement	AES-256-GCM	
Dérivation de clé depuis MDP maître	PBKDF2-SHA256, 100 000 itérations	
Protection de la clé en mémoire	Jamais stockée — recalculée à chaque session	
Authentification du coffre (intégrité)	HMAC-SHA256 sur le vault chiffré	
Protection anti-brute-force	Blocage + délai exponentiel après échecs	
Stockage persistant sécurisé	Vault chiffré sur serveur + local	
Audit de sécurité externe	Audits publics (2018, 2020, 2022)	
Open source (code vérifiable)	Oui — GitHub public	

PARTIE 20 — AUDIT DE SÉCURITÉ DU PROJET (~60 min)

QU'EST-CE QU'UN AUDIT DE SÉCURITÉ ?

Un audit de sécurité (pentest = penetration test) consiste à analyser un système du point de vue d'un attaquant pour identifier ses vulnérabilités AVANT qu'un vrai attaquant ne les exploite.

Le rapport d'audit liste les vulnérabilités avec :

- Une description technique précise
- Une évaluation de la criticité (score CVSS : 0 à 10)
- Les conditions d'exploitation
- Une recommandation de correction (contre-mesure)

Votre mission : vous mettre dans la peau d'un auditeur de sécurité et rédiger le rapport d'audit du gestionnaire de mots de passe Arduino.

20.1 Grille d'évaluation CVSS simplifiée

Score CVSS	Niveau	Accès requis	Conséquence	Exemples
9.0 – 10.0	CRITIQUE	Réseau / sans accès physique	Compromission totale	Exécution de code à distance
7.0 – 8.9	ÉLEVÉ	Accès local ou conditionnel	Fuite de données importantes	Cassage du chiffrement
4.0 – 6.9	MOYEN	Accès physique requis	Fuite partielle	Récupération du coffre
0.1 – 3.9	FAIBLE	Conditions très spécifiques	Impact limité	Déni de service local

20.2 Rapport d'audit — À rédiger

Identifiez et documentez les vulnérabilités du gestionnaire de mots de passe Arduino. Vous devez trouver au moins 6 vulnérabilités réelles. Certaines vous sont suggérées, d'autres sont à trouver par votre analyse.

#	Description de la vulnérabilité	Score /10	Conditions d'exploitation	Impact si exploitée	Contre-mesure recommandée
V1	Clé secrète stockée en clair dans le code source (cleSecrete = 'SECRET2025')	/10			
V2	Algorithme de chiffrement XOR — vulnérable aux attaques KPA et analyse de fréquence	/10			
V3		/10			
V4		/10			

QUESTION CY7

[illegible][illegible]

BILAN DU TP

Compétence évaluée	Auto-évaluation	Évaluation professeur
Je sais identifier les composants électroniques du projet	☐ Acquis ☐ En cours	/5
Je sais calculer une résistance avec la loi d'Ohm	☐ Acquis ☐ En cours	/5
J'ai réalisé le câblage complet et fonctionnel sur Tinkercad	☐ Acquis ☐ En cours	/5
Je comprends le principe du clavier matriciel	☐ Acquis ☐ En cours	/5
Je comprends le protocole I2C et son avantage	☐ Acquis ☐ En cours	/5
J'ai su dépanner les problèmes rencontrés	☐ Acquis ☐ En cours	/5
Je sais dessiner un schéma normalisé IEC	☐ Acquis ☐ En cours	/5
Je sais utiliser le multimètre et l'oscilloscope virtuels	☐ Acquis ☐ En cours	/5
Je sais calculer la puissance et l'autonomie d'un système	☐ Acquis ☐ En cours	/5
Je sais lire et annoter du code Arduino existant	☐ Acquis ☐ En cours	/5
J'ai rédigé un compte-rendu technique complet	☐ Acquis ☐ En cours	/5
J'ai proposé des améliorations matérielles justifiées	☐ Acquis ☐ En cours	/5
Je maîtrise les types de données C/C++ sur Arduino	☐ Acquis ☐ En cours	/5
Je sais écrire et corriger des fonctions Arduino	☐ Acquis ☐ En cours	/5
Je sais déboguer avec le Serial Monitor	☐ Acquis ☐ En cours	/5

BAC PRO CIEL — TP ÉLECTRONIQUE : Gestionnaire de Mots de Passe		TP Électronique
Compétence évaluée	Auto-évaluation	Évaluation professeur
Je maîtrise les conversions binaire / hex / ASCII	☐ Acquis ☐ En cours	/5
J'ai réalisé les attaques KPA et analyse de fréquence sur XOR	☐ Acquis ☐ En cours	/5
Je comprends le hachage, le sel et AES-256	☐ Acquis ☐ En cours	/5
J'ai rédigé un rapport d'audit de sécurité complet	☐ Acquis ☐ En cours	/5

Commentaires du professeur :

 **FIN DU PROJET** 

Félicitations